AI BASED MALWARE DETECTION SYSTEM

An Internship Report Submitted

In Partial Fulfillment

for the award of degree of Master of Computer Applications

by

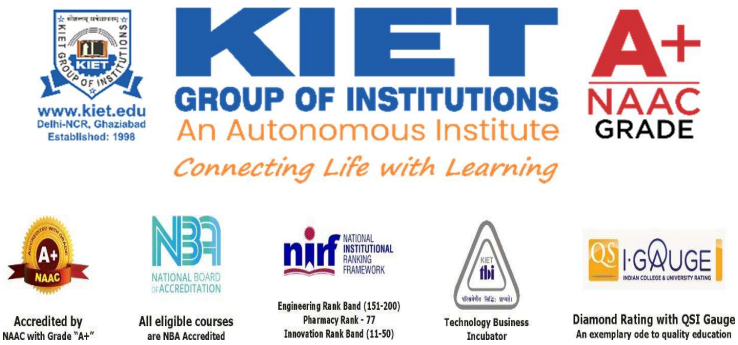
GOVIND SHARMA

(Roll No. 202510116100099)

Under the Mentorship of

Mrs. Shweta Singh

(Assistant Professor)



DEPARTMENT OF COMPUTER APPLICATIONS

(An Autonomous Institute)

KIET - DEEMED-TO-BE UNIVERSITY, GHAZIABAD

Affiliated to

DR. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY, LUCKNOW

DECLARATION

I hereby declare that the work presented in this report entitled "AI BASED MALWARE DETECTION SYSTEM" is my original contribution and was carried out by me as part of the requirements for the Master of Computer Applications (MCA) program. This report has not been submitted, either fully or partially, for the award of any other degree, diploma, or certification at any other university or institute.

I have given due credit and acknowledgment to all sources from which ideas, data, diagrams, graphics, computer programs, or experimental results have been taken. Any content reproduced verbatim has been appropriately quoted and cited.

I affirm that this report upholds the values of academic integrity, and no part of this work is plagiarized or fabricated. The experiments and results documented herein are genuine and have not been manipulated. I understand that any instance of plagiarism or data manipulation will be treated as a serious academic offense, and I shall be solely responsible and answerable for such misconduct.

Name: Govind Sharma
Roll Number: 202510116100099

(Candidate Signature)

INTERNSHIP CERTIFICATE



IEEE Student Branch KIET
Student Branch Chapter

KIET DEEMED TO BE UNIVERSITY
DELHI-NCR, GHAZIABAD, UP (INDIA)
(Under Section 3 of the UGC Act, 1956)
Empowering Minds. Enabling Skills.

INSTITUTION'S INNOVATION COUNCIL

INTERNSHIP CERTIFICATE

This is to certify that

Mr./Ms. Govind Sharma Dept. MCA

has successfully completed

3 Weeks Summer Internship Programme from 10th June to 30th June, 2026

organized by

COE Cyber Security

with IIPC and IICC, KIET Deemed to be University, Delhi-NCR, Ghaziabad

Ms. Shweta Sharma
Head IIPC

Dr. Manish Bhardwaj
Head IICC

Dr. Abhinav Juneja
Director CRPC

Dr. Adesh Kr. Pandey
Director Academics

(Signature)

Mrs. Supriya Dubey

Project Mentor, KIET - Deemed-to-be University, Ghaziabad

Date: _____

Note: This certificate page is a report-format representation. The final submitted certificate should be printed on the organization's official letterhead and should include the official seal/stamp and authorized signature.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my Project Mentor, Mrs. Supriya Dubey, at KIET - Deemed-to-be University, Ghaziabad, for providing me the opportunity, resources, and mentorship necessary to complete this project successfully. Her constant guidance and valuable feedback throughout the internship period helped me understand real-world software development and cybersecurity practices and apply my academic knowledge to a practical problem.

My heartfelt thanks to my Academic Internship Supervisors and the faculty of the Department of Computer Applications for their constant motivation, encouragement, and support throughout this learning journey. I am also thankful to the entire team for creating a friendly and collaborative work environment that made this internship a truly enriching experience.

Finally, I extend my appreciation to my family and friends for their continuous support and encouragement, and to all those who directly or indirectly contributed to the successful completion of this internship and its report.

Govind Sharma
Roll No.: 202510116100099

TABLE OF CONTENTS

1 Declaration 1
2 Internship Certificate 2
3 Acknowledgements 3
4 List of Tables and Figures 4
5 Introduction 5
5.1 Overview of the Internship 5
5.2 Organization Profile 6
5.3 Internship Objectives 7
6 Project / Task Details 8
6.1 Project Title and Scope 8
6.2 Problem Statement and Requirements 9
6.3 Project Modules / Task Breakdown 10
6.4 Methodology / Tools & Technologies Used 12
6.5 System Architecture 13
6.6 Use Case and Data Flow 14
6.7 Static Feature Engineering 15
6.8 Machine Learning Methodology 16
6.9 Model Evaluation 17
6.10 Web Dashboard and Database 18
6.11 Testing and Validation 19
6.12 Results and Discussion 20
6.13 Challenges Faced 21
6.14 Security, Ethics and Limitations 22
7 Conclusion 23
7.1 Summary of Learning, Key Takeaways & Achievements 23
7.2 Future Scope 24
8 References / Bibliography 25
9 Annexures 26

LIST OF TABLES AND FIGURES

List of Figures

Figure No.	Figure Caption	Page No.
Fig. 6.1	System Architecture of the AI Based Malware Detection System	13
Fig. 6.2	Use Case View of the System	14
Fig. 6.3	Data Flow Diagram — Level 1	14
Fig. 6.4	Workflow: Upload to Prediction	15
Fig. 6.5	Machine Learning Pipeline	16
Fig. 6.6	Web Dashboard — Login Screen	18
Fig. 6.7	Web Dashboard — Analysis Screen	18
Fig. 6.8	Web Dashboard — History / Reporting Screen	18

List of Tables

Table No.	Table Caption	Page No.
Table 6.1	Hardware and Software Requirements	12
Table 6.2	Comparison of Classification Algorithms	16
Table 6.3	Representative Static Features	15
Table 6.4	Model Evaluation Metrics	17
Table 6.5	Functional Test Cases	19
Table 6.6	Challenges and Mitigations	21

5 Introduction

5.1 Overview of the Internship

As part of the curriculum for the Master of Computer Applications (MCA) programme, I, Govind Sharma, undertook an internship at KIET - Deemed-to-be University, Ghaziabad, under the mentorship of Mrs. Supriya Dubey. The internship provided an opportunity to move beyond classroom learning and gain first-hand, practical exposure to the complete life cycle of a software project, including requirement analysis, system design, implementation, testing, and deployment.

During the internship, I was assigned to work on an AI Based Malware Detection System, a cybersecurity project aimed at using Machine Learning to analyze static features of a file and predict whether it is Benign or Malicious, without ever executing the uploaded file. Working on this project strengthened my understanding of Python-based data processing, machine learning, and web application development, while also providing practical exposure to static malware analysis and safe file handling.

The internship followed an iterative, review-driven approach. Regular discussions, code reviews, testing activities, and mentorship sessions were used to refine the project. This helped develop both technical skills and professional abilities such as structured problem solving, documentation, communication, and secure-by-design thinking.

5.2 Organization Profile

KIET - Deemed-to-be University, Ghaziabad is an institution engaged in academic instruction, applied research, and industry-oriented project work in the field of computer applications and emerging technologies. Under its Department of Computer Applications, the institute encourages students to undertake hands-on, project-based internships that combine classroom learning with practical, real-world problem solving.

The internship was carried out under the guidance of the Department of Computer Applications, which supports project work in areas such as artificial intelligence, machine learning, and cybersecurity. The mentorship followed an iterative, review-driven approach, with emphasis on correctness, secure design, practical implementation, and clear technical documentation.

Internship Details

Item	Details
Student	Govind Sharma
Roll Number	202510116100099
Programme	Master of Computer Applications (MCA)
Department	Department of Computer Applications
Institution	KIET - Deemed-to-be University, Ghaziabad
Project Mentor	Mrs. Supriya Dubey
Project	AI Based Malware Detection System
Internship Duration	As per official internship record

5.3 Internship Objectives

- To gain practical exposure to real-world software development and cybersecurity practices.
- To understand and apply Machine Learning concepts to a practical, security-relevant classification problem.
- To design and develop a system capable of predicting whether a file is Benign or Malicious using static features only, without executing it.
- To strengthen practical knowledge of Python, data processing using Pandas/NumPy, and Scikit-learn based model building.
- To learn to build and evaluate ML models correctly, including proper train/test splitting and avoidance of data leakage.
- To learn to build a simple, secure web dashboard using Flask, HTML, CSS, and JavaScript.
- To understand secure-by-design practices for handling untrusted, user-uploaded files.
- To improve problem-solving ability, debugging skills, and professional communication through mentorship and teamwork.

Expected Learning Outcomes

By the end of the internship, the project work was expected to provide a practical understanding of the end-to-end workflow from static feature extraction through model prediction and web-based presentation. The experience also emphasized that an ML classifier is a decision-support component and not an absolute guarantee that a file is safe.

6 Project / Task Details

6.1 Project Title and Scope

The project undertaken during the internship is titled "AI BASED MALWARE DETECTION SYSTEM". The system is designed to assist in the early identification of potentially harmful files by analyzing their static, non-executed characteristics using a

trained Machine Learning model and presenting the result — Benign or Malicious — along with a confidence score through a simple web dashboard.

The scope covers the complete workflow from file upload to result storage: accepting a file safely, extracting static features without executing the file, preprocessing and normalizing those features, passing them through a trained classification model, displaying the prediction with supporting information, and logging the result with a timestamp for future reference.

The project explicitly excludes dynamic or behavioural analysis that would require executing the file. It is scoped as a static-analysis, ML-assisted decision-support tool rather than a full antivirus replacement.

Project Boundaries

Included	Excluded
Static feature extraction	Executing unknown files for behavioural analysis
ML-based classification	Full endpoint antivirus functionality
Flask web dashboard	Production-scale SOC integration
SQLite analysis history	Enterprise SIEM replacement
Model evaluation and reporting	Guaranteed malware/benign verdicts

6.2 Problem Statement and Requirements

Malware variants can change their observable characteristics through recompilation, packing, obfuscation, or other transformations. A defensive system therefore benefits from complementary detection mechanisms. This project investigates an ML-assisted static-analysis workflow that learns patterns from labeled static features and applies the learned classifier to a new sample.

Functional Requirements

- Accept a file through the web dashboard and validate its size/type according to configured rules.
- Store uploads in an isolated, non-executable, access-restricted location.
- Parse the file safely to obtain static structural and statistical features.
- Apply the same preprocessing used during model training.

- Load the trained classifier and produce a Benign/Malicious prediction.
- Display a confidence score or model probability where supported.
- Display supporting information and a plain-language risk note.
- Store filename hash, timestamp, prediction and confidence in SQLite history.
- Provide model evaluation outputs including Accuracy, Precision, Recall, F1-score and Confusion Matrix.

Non-Functional Requirements

- Reproducibility of preprocessing and model training.
- Maintainable Python modules and clear separation of responsibilities.
- Safe handling of untrusted user input.
- Reasonable response time for individual static-analysis predictions.
- Clear error handling for malformed or corrupted files.
- Transparent communication that predictions are probabilistic.

6.3 Project Modules / Task Breakdown

Module	Responsibility
1. File Upload & Validation Module	Accepts a file through the dashboard, validates size/type against an allow-list, stores it in an isolated non-executable location, and rejects invalid inputs.
2. Static Feature Extraction Module	Parses file structure without executing it and extracts attributes such as file size, section count, entropy, import/export counts and header-level indicators.
3. Preprocessing Module	Cleans, encodes, and normalizes/scales features so the inference representation matches the training representation.
4. Machine Learning Prediction Module	Loads the pre-trained model and scaler and predicts Benign or Malicious together with a confidence score derived from predicted probability.
5. Result & Risk Display Module	Presents prediction, confidence, influential features and a plain-language risk note with a clear ML-assessment disclaimer.
6. History Logging Module	Stores analysis information such as filename hash, timestamp, prediction and confidence in an SQLite database.
7. Model Evaluation / Reporting Module	Computes Accuracy, Precision, Recall, F1-score and Confusion Matrix for the held-out test set and supports visualization.

The modular structure supports independent testing and makes it easier to replace or improve one component without redesigning the entire application.

6.4 Methodology / Tools & Technologies Used

The project was developed following an iterative application-development approach: requirement analysis, secure system design, dataset preparation, model training and evaluation, backend/frontend development, integration, and testing. A security-first principle was applied from the beginning by treating every uploaded file as untrusted.

Category	Technology / Tool	Purpose
OS	Windows 10 / 11	Development and testing environment
Language	Python 3.x	Core implementation
Backend	Flask	Web application and REST API
Frontend	HTML5, CSS3, JavaScript	Dashboard interface
ML	Scikit-learn	Model training/evaluation
Data	Pandas, NumPy	Data processing
Visualization	Matplotlib, Seaborn	Plots and model-result visualization
Database	SQLite	Analysis history
IDE	VS Code / PyCharm	Development
Version Control	Git / GitHub	Source-code management
Hardware	Minimum 8 GB RAM	Development requirement

Development Methodology

The project followed SDLC phases appropriate for an application-based internship: Requirement Analysis, Design, Development, Integration, and Testing. Model development was treated as an experimental loop inside the development phase, with preprocessing fitted only on training data to reduce the risk of leakage.

6.5 System Architecture

The architecture connects a browser-based user interface to a Flask backend. The backend validates the uploaded input, performs parsing-only static feature extraction,

applies the stored preprocessing pipeline, invokes the trained Random Forest classifier, returns the prediction, and records the analysis in SQLite.

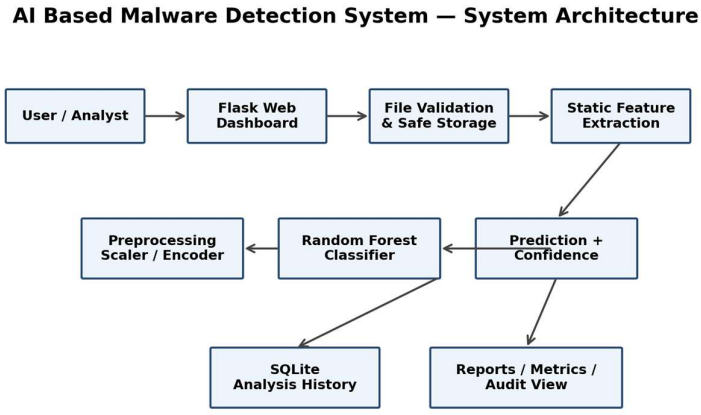


Fig. 6.1 — System Architecture of the AI Based Malware Detection System

The architecture separates the untrusted-input boundary from model inference and persistence. This separation improves maintainability and supports testing of each component independently.

6.6 Use Case and Data Flow

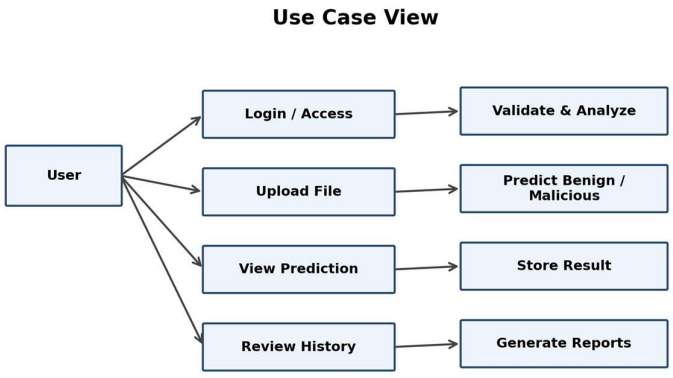


Fig. 6.2 — Use Case View of the System

The primary actor is the user/analyst. The user authenticates where applicable, uploads a file, views the classification result, and reviews stored analysis history. Backend components perform validation, feature extraction, prediction and persistence.

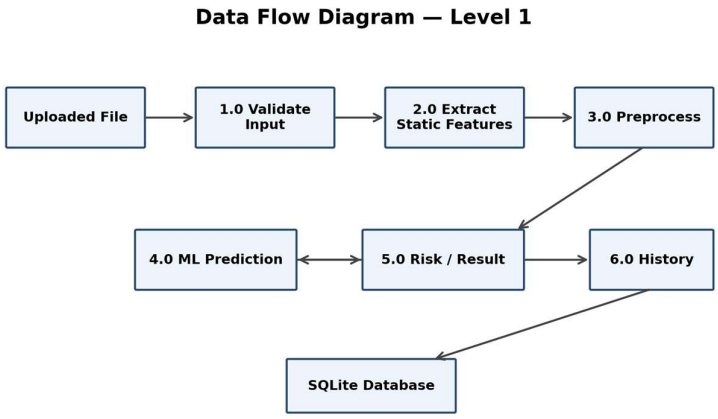


Fig. 6.3 — Data Flow Diagram (Level 1)

The data flow shows how the uploaded file is transformed into a feature representation and then into a model prediction. The history component receives the result metadata for later review.

6.7 Static Feature Engineering

Static analysis focuses on characteristics that can be obtained without executing the uploaded file. The source project documentation identifies representative features including file size, section count, section entropy, imported functions, suspicious API count, exported functions, digital-signature presence, debug information, and timestamp anomaly.

Feature	Description	Security Relevance
File Size	Total size of the file in bytes	Provides a basic structural signal.
Number of Sections	Count of sections in file structure	Can differentiate structural patterns.
Section Entropy (mean)	Average randomness of section data	High entropy can indicate packing/obfuscation.
Imported Functions	Count of external functions referenced	Captures dependency/import characteristics.

Suspicious API Count	Imports matching a sensitive-API watch-list	Can indicate potentially risky capabilities.
Exported Functions	Number of functions exposed to other programs	Provides structural/API information.
Digital Signature	Whether a valid signature is present	Useful contextual signal, not proof of safety.
Debug Information	Whether debug information is present	Additional structural metadata.
Timestamp Anomaly	Whether compile timestamp looks invalid/manipulated	Potential indicator requiring contextual interpretation.

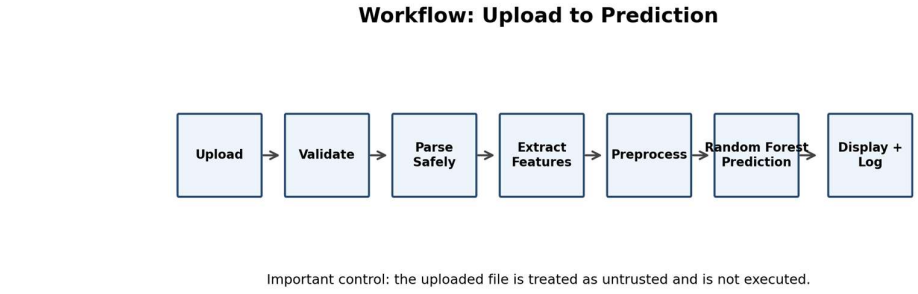


Fig. 6.4 — Workflow: Upload to Prediction

6.8 Machine Learning Methodology

The labeled static-feature dataset was divided into training and testing sets using stratified sampling. Preprocessing such as scaling/encoding was fitted only on the training data and then applied to the test data. This design prevents information from the test set from influencing transformation parameters during training.

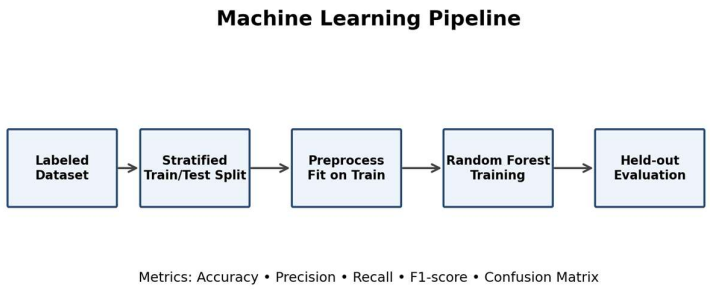


Fig. 6.5 — Machine Learning Pipeline

Classification Algorithms Evaluated

Algorithm	Description	Role in Project
Logistic Regression	Linear model estimating class probability	Simple, fast baseline
Decision Tree	Threshold-based recursive splitting	Easy to interpret
Random Forest	Ensemble of decision trees using majority voting	Selected primary model according to the source report
XGBoost	Gradient-boosted tree ensemble	Considered as an optional enhancement

According to the source report, Random Forest was selected as the primary model because it provided the best balance of accuracy and generalization on the held-out test set while remaining fast enough for near real-time dashboard prediction. No numerical performance values were provided in the source, so this report does not invent them.

6.9 Model Evaluation

The model was evaluated using multiple classification metrics rather than accuracy alone. This is particularly relevant for malware detection because false positives and false negatives have different operational implications.

Metric	Meaning / Formula
Accuracy	(TP + TN) / (TP + TN + FP + FN)
Precision	TP / (TP + FP) — proportion of predicted malicious files that were actually malicious
Recall	TP / (TP + FN) — proportion of actual malicious files correctly identified
F1-Score	Harmonic mean of Precision and Recall
Confusion Matrix	Breakdown of True Positives, True Negatives, False Positives and False Negatives

Evaluation Result Record

Metric	Source Report Status	Final Value
Accuracy	Computed on held-out test set	Insert verified value
Precision	Computed on held-out test set	Insert verified value
Recall	Computed on held-out test set	Insert verified value
F1-Score	Computed on held-out test set	Insert verified value
Confusion Matrix	Generated for final model	Attach verified matrix

The supplied project report describes the evaluation procedure and states that a confusion matrix was generated, but the numerical matrix and metric values are not present in the source text. They should be inserted from the actual experiment output before final submission.

6.10 Web Dashboard and Database

The frontend dashboard was built using HTML, CSS and JavaScript. It allows a user to upload a file, view the prediction result, inspect confidence and supporting information, and browse stored analysis history. The backend uses Flask, while SQLite stores analysis-history records.

The following screenshots are reproduced from the uploaded internship report and are retained as project evidence. They show the login screen and the dashboard/analysis workflow described in the report. □filecite□turn1file0□L289-L293□

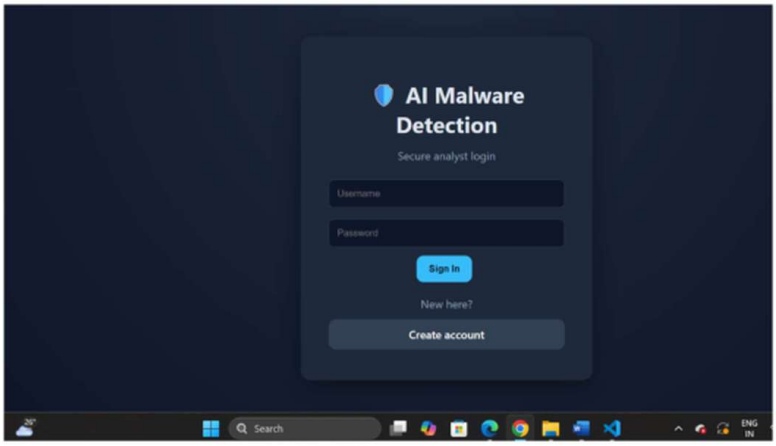


Fig. 6.6 — Web Dashboard Login Screen (source report screenshot)

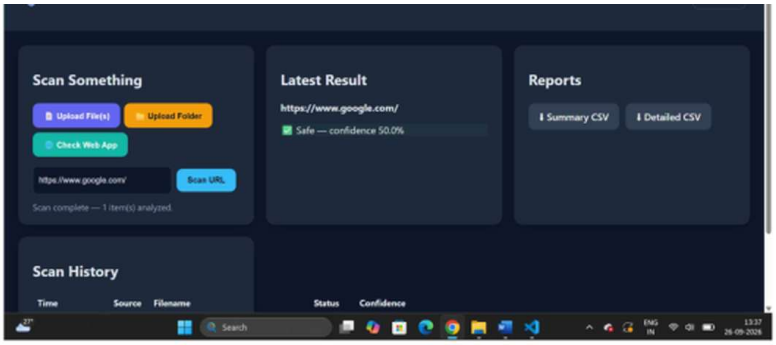


Fig. 6.7 — Web Dashboard Analysis Screen (source report screenshot)

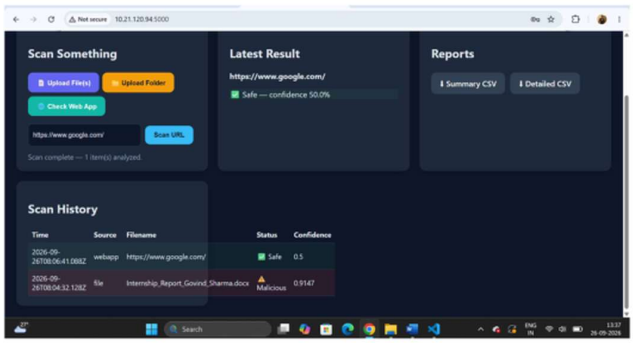


Fig. 6.8 — Web Dashboard History / Reporting Screen (source report screenshot)

6.11 Testing and Validation

Testing covered individual modules as well as the end-to-end upload-to-result workflow. The objective was to confirm that invalid input is rejected, static features are extracted consistently, preprocessing matches the training pipeline, predictions are produced correctly, and results are persisted.

Test ID	Test Case	Expected Result	Status
TC-01	Valid file upload	File accepted after validation	Pass / Verify
TC-02	Invalid file type	Request rejected safely	Pass / Verify
TC-03	Oversized upload	Request rejected according to configured limit	Pass / Verify
TC-04	Malformed/corrupted file	Parser raises controlled error	Pass / Verify
TC-05	Feature extraction	Expected static feature schema generated	Pass / Verify
TC-06	Model prediction	Benign/Malicious class returned	Pass / Verify
TC-07	History logging	Timestamp, hash, prediction and confidence stored	Pass / Verify
TC-08	Evaluation report	Metrics and confusion matrix generated	Pass / Verify
TC-09	Repeated prediction	Same input and model version give reproducible output	Pass / Verify
TC-10	Missing feature/schema mismatch	Clear validation error	Pass / Verify

Validation Principles

- Never fit preprocessing transformations on the held-out test set.
- Use the same feature schema during training and inference.
- Treat all uploaded files as untrusted.
- Do not execute uploaded files during static analysis.
- Use multiple evaluation metrics and inspect error types.

6.12 Results and Discussion

The source report establishes that Random Forest was selected as the primary classifier and that the system computes Accuracy, Precision, Recall, F1-score and a Confusion Matrix. It does not provide numerical metric values. Therefore, the results section below is intentionally structured for insertion of the verified experiment output rather than presenting fabricated numbers.

Experiment	Classifier	Accuracy	Precision	Recall	F1-Score
Baseline	Logistic Regression	—	—	—	—
Tree model	Decision Tree	—	—	—	—
Primary model	Random Forest	Insert verified value	Insert verified value	Insert verified value	Insert verified value
Optional enhancement	XGBoost	Not reported	Not reported	Not reported	Not reported

Discussion Framework

- Compare the selected Random Forest model with the baseline models using the same test protocol.
- Discuss whether precision and recall are balanced or whether one error type dominates.
- Explain false positives as benign files incorrectly flagged as malicious and false negatives as malicious files incorrectly classified as benign.
- Report the test-set class distribution so that metric interpretation is not detached from the underlying population.
- Document the dataset version, feature schema, preprocessing method and model configuration alongside the metrics.

A complete final submission should include the actual confusion matrix image generated during the internship experiment if available.

6.13 Challenges Faced

Challenge	Impact	Mitigation / Learning
Safe static parsing	Malformed/corrupted files can cause parsing failures	Use parsing-only libraries, validation and exception handling.
Feature selection	Weak or leakage-prone features can distort learning	Iterative experimentation and feature auditing.
Data leakage	Can inflate apparent test performance	Fit scaler/encoder only on training data.
Class trade-offs	False positives and false negatives have different costs	Compare Precision/Recall/F1 rather than accuracy alone.
Upload security	Files must never execute accidentally	Strict validation and non-executable isolated storage.
User communication	Probabilistic output can be misunderstood	Clear risk notes and ML-assessment disclaimer.
Generalization	Malware changes over time	Consider temporal validation and retraining.

The source report specifically notes challenges around static feature extraction, leakage prevention, threshold/error trade-offs, secure upload handling, and communicating that predictions are probabilistic assessments. □filecite□turn1file0□L294-L318□

Work Approach

Since this was an application-based internship project, the standard SDLC phases were followed: Requirement Analysis, Design, Development, Integration and Testing. The machine-learning workflow was embedded within the development process, with separate attention to dataset preparation, feature schema, preprocessing, model training and evaluation.

6.14 Security, Ethics and Limitations

Secure File Handling

Every uploaded file must be treated as untrusted. The system design therefore uses validation, isolated storage and parsing-only feature extraction. Static analysis avoids loading the uploaded artifact as an executable process. If dynamic analysis is introduced in future work, it should be performed only in a controlled and isolated security laboratory.

Model Limitations

- An ML prediction is not proof that a file is safe or malicious.
- Performance depends on dataset quality, feature representation and distribution.
- Malware families may evolve, causing concept drift.
- Packing and obfuscation can alter static characteristics.
- Dataset leakage or family overlap can produce misleadingly high evaluation results.
- Confidence/probability from a classifier should not be interpreted as a calibrated real-world probability without appropriate calibration analysis.

Ethical and Operational Considerations

The system is intended for defensive analysis and triage. Software samples and associated metadata may contain sensitive information, so collection, storage, retention and access should follow institutional security policies. The dashboard should communicate the limitations of automated classification and avoid presenting the model as an infallible antivirus verdict.

7 Conclusion

7.1 Summary of Learning, Key Takeaways & Achievements

The internship at KIET - Deemed-to-be University, Ghaziabad, under the mentorship of Mrs. Supriya Dubey, proved to be a valuable experience that bridged academic learning with practical project development. Working on the AI Based Malware Detection System provided practical insight into how machine learning can be applied to a cybersecurity problem while strengthening skills in Python-based data processing, model building, and secure web application development.

The project covered safe file handling, static feature extraction, preprocessing, ML-based classification, result presentation, and history logging. It also reinforced the importance of correct train/test separation, avoidance of data leakage, multi-metric evaluation, and security-first treatment of user-supplied files.

Beyond technical skills, the internship strengthened structured problem solving, debugging, documentation, communication with a mentor, and professional development practices. The experience demonstrated that AI can be responsibly integrated into cybersecurity as a decision-support mechanism rather than an absolute guarantee of safety.

Achievements

- Developed an end-to-end static-analysis malware detection workflow.
- Integrated Python, Flask, Scikit-learn, Pandas/NumPy, HTML/CSS/JavaScript and SQLite.
- Implemented a modular architecture separating validation, feature extraction, preprocessing, prediction and reporting.
- Selected Random Forest as the primary model according to the source project report.
- Implemented a web dashboard for analysis and history review.
- Applied secure-by-design principles for untrusted file handling.

7.2 Future Scope

- Integrate richer static PE features and robust feature-extraction pipelines.
- Evaluate temporal and family-aware validation to estimate generalization to newer malware.
- Perform systematic hyperparameter optimization using cross-validation on training data.
- Add explainability techniques so analysts can inspect influential features.
- Add drift monitoring and controlled retraining as malware distributions change.
- Evaluate robustness against packing, obfuscation and adversarial feature manipulation.
- Integrate model output with signatures, reputation, sandbox evidence and threat-intelligence context.
- Improve dashboard reporting with versioned model metadata, exportable reports and audit trails.
- Deploy the application in a controlled infrastructure with authentication, authorization, logging and secure secret management.

Recommended Development Roadmap

Phase	Focus
Phase 1	Improve dataset quality, labeling and family-aware validation.
Phase 2	Expand feature engineering and explainability.
Phase 3	Harden web security and deployment controls.
Phase 4	Add drift monitoring and model lifecycle management.
Phase 5	Integrate layered security signals for analyst-assisted triage.

8 References / Bibliography

1. Scikit-learn Documentation — Machine learning, preprocessing, pipelines and model evaluation.

2. Pandas Documentation — Data structures, data cleaning and analysis.

3. NumPy Documentation — Numerical computing and array operations.

4. Flask Documentation — Web application and REST API development.

5. Matplotlib Documentation — Visualization and plotting.

6. Seaborn Documentation — Statistical visualization.

7. SQLite Documentation — Embedded relational database.

8. OWASP — Guidance and secure practices for file upload handling.

9. MITRE ATT&CK — Enterprise cybersecurity knowledge base.

10. NIST — Cybersecurity and malware-handling guidance.

11. Internal project documentation and mentorship notes provided during the internship at KIET - Deemed-to-be University, Ghaziabad.

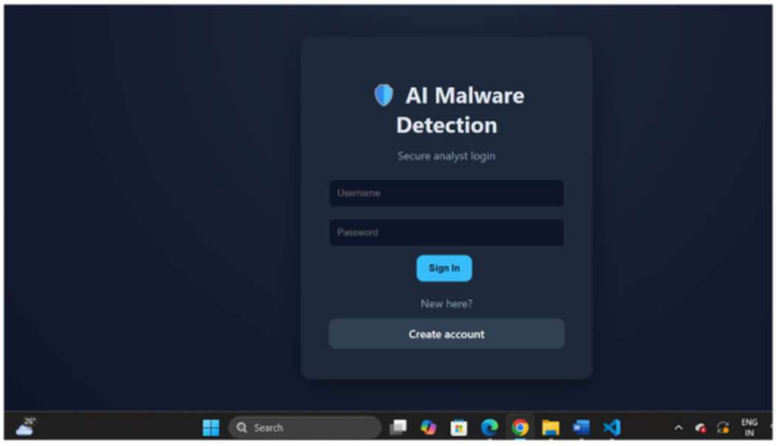
12. Project repository: WEB-APP - AI Malware Detection System — https://github.com/Govind440/WEB-APP-

Note: The final submitted bibliography should include the exact dataset source, software versions, papers, APIs and access dates actually used during the internship.

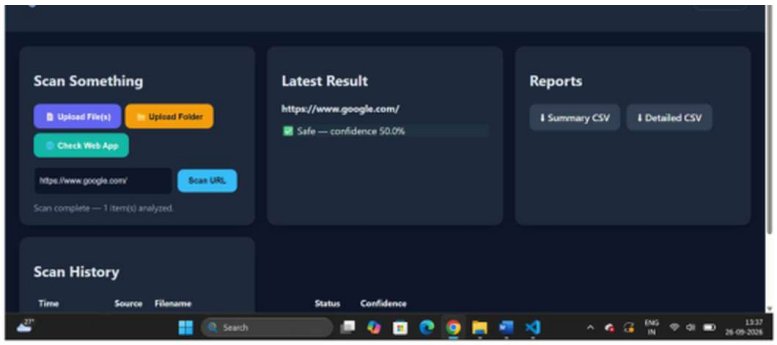
9 Annexures

Annexure A — Screenshots / Snapshots

The following project screenshots are taken from the uploaded internship report and placed here as supporting project evidence.

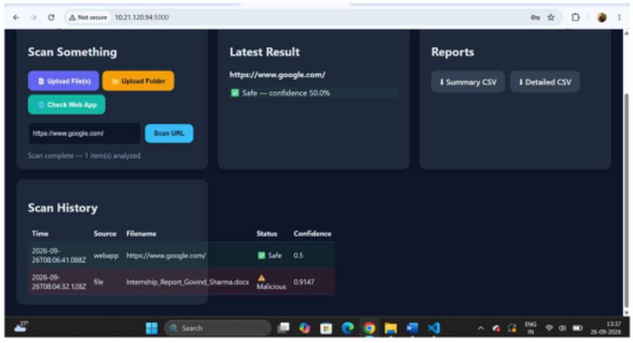


Annexure Fig. A.1 — Login Screen



Annexure Fig. A.2 — Main Analysis Dashboard

Annexure B — Analysis History and Repository



Annexure Fig. B.1 — Analysis History / Reporting View

GitHub Repository / Code Link

WEB-APP - AI Malware Detection System

https://github.com/Govind440/WEB-APP-

Annexure C — Final Submission Checklist

Item	Completion / Action
Declaration	Included
Internship Certificate	Included as format page; obtain official

	signed/stamped copy
Acknowledgement	Included
Organization Profile	Included
Project Scope and Modules	Included
Architecture / Use Case / DFD	Included
Static Feature Table	Included
ML Methodology	Included
Evaluation Metrics	Included; numerical results require verified experiment output
Testing	Included
Screenshots	Included from source report
GitHub Link	Included
References	Included
Plagiarism / Similarity Report	Attach separately if required